

GHANA COMMUNICATION TECHNOLOGY UNIVERSITY



FALCUTY OF COMPUTING AND INFORMATION SYSTEMS

PROGRAM: SOFTWARE ENGINEERING

COURSE TITLE: CSSM 421

SOFTWARE MODELING AND ANALYSIS

NAME: BENJAMIN BAIDOO

INDEX NUMBER: 4231230141

Table of Contents

System Overview	1
Use Case Diagram For Canteen Management System	1
Class Diagram For The Canteen Management System	2
Entity Relationship Diagram for the canteen management system	5
Sequence Diagram For The Canteen Management System.....	7

Table of figures

FIGURE 1 USE CASE DIAGRAM	2
FIGURE 2 CLASS DIAGRAM	5
FIGURE 3 ENTITY RELATIONSHIP DIAGRAM	6
FIGURE 4 SEQUENCE DIAGRAM	8

System Overview

The School Canteen Management System is designed to automate manual food ordering and payment processes to address issues such as long queues and lost records. The system is a centralized web-based application supporting Students, Staff, Cashiers, Administrators, and Distribution Officers. It ensures accountability through tracking mechanisms and provides management with daily and monthly revenue reports

Use Case Diagram For Canteen Management System

Design Explanation & Justification

This diagram visualizes the interactions between the five identified actors and the system.

- **Actors:** All required actors are represented: Administrator, Student, Staff, Cashier, and Distribution Officer.
- **Shared Use Cases:** Both **Student** and **Staff** actors are connected to the "Place Food Order" use case, as the system description explicitly notes that Staff perform this action just like students.
- **Automated Actions (<<include>>):** The requirement states the system must "Automatically send a text message (SMS)... when an order is paid". In UML, this is best modelled as an **Include** relationship. The "Confirm Payment" use case *includes* the "Send SMS Notification" use case, showing that the SMS is a mandatory, automatic part of the payment process, not a separate manual action by the Cashier.
- **Distribution Officer:** This actor is modelled specifically to "Confirm Collection", ensuring the physical handover of food is tracked in the system.

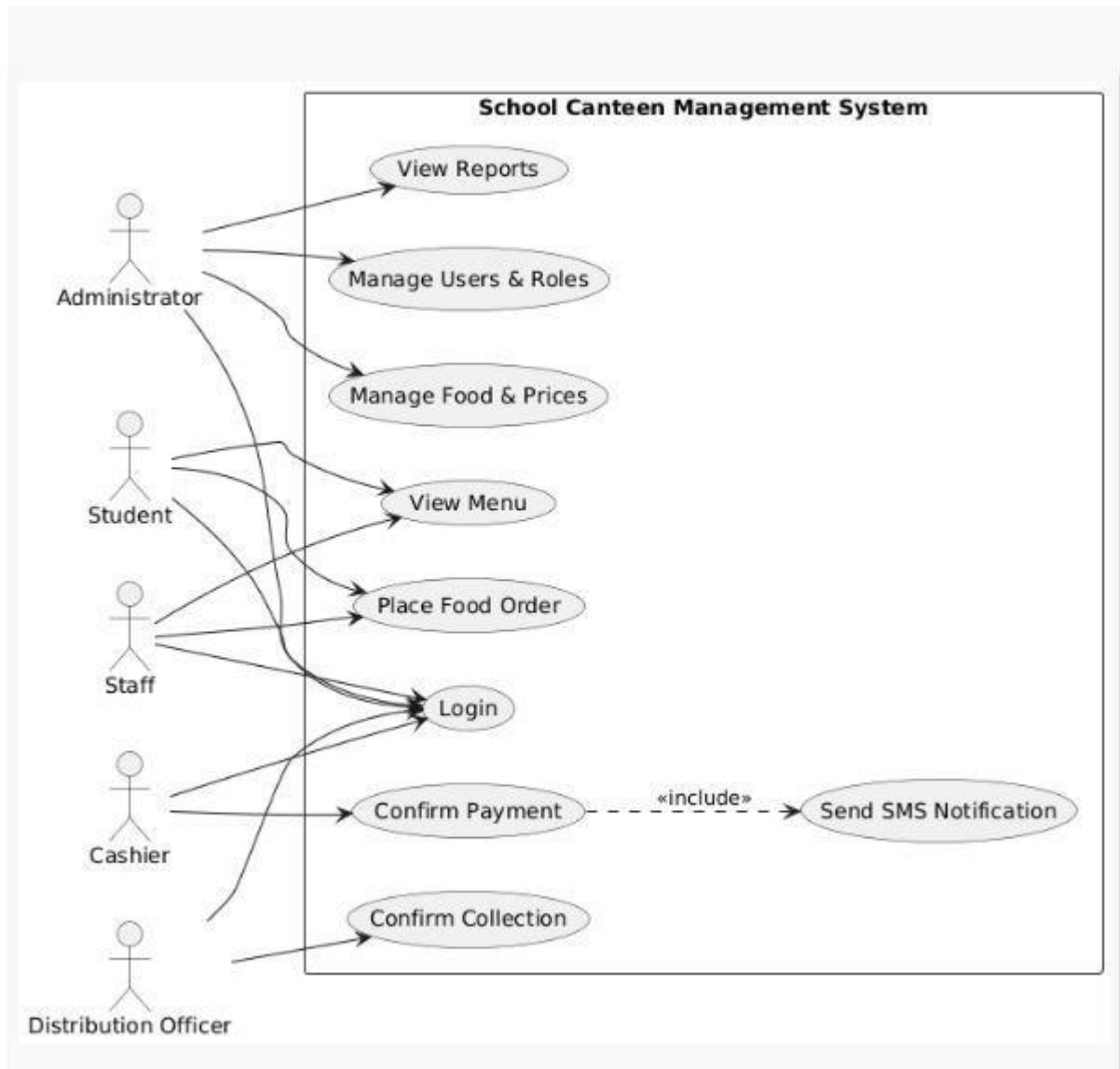


Figure 1 Use Case Diagram

Class Diagram For The Canteen Management System

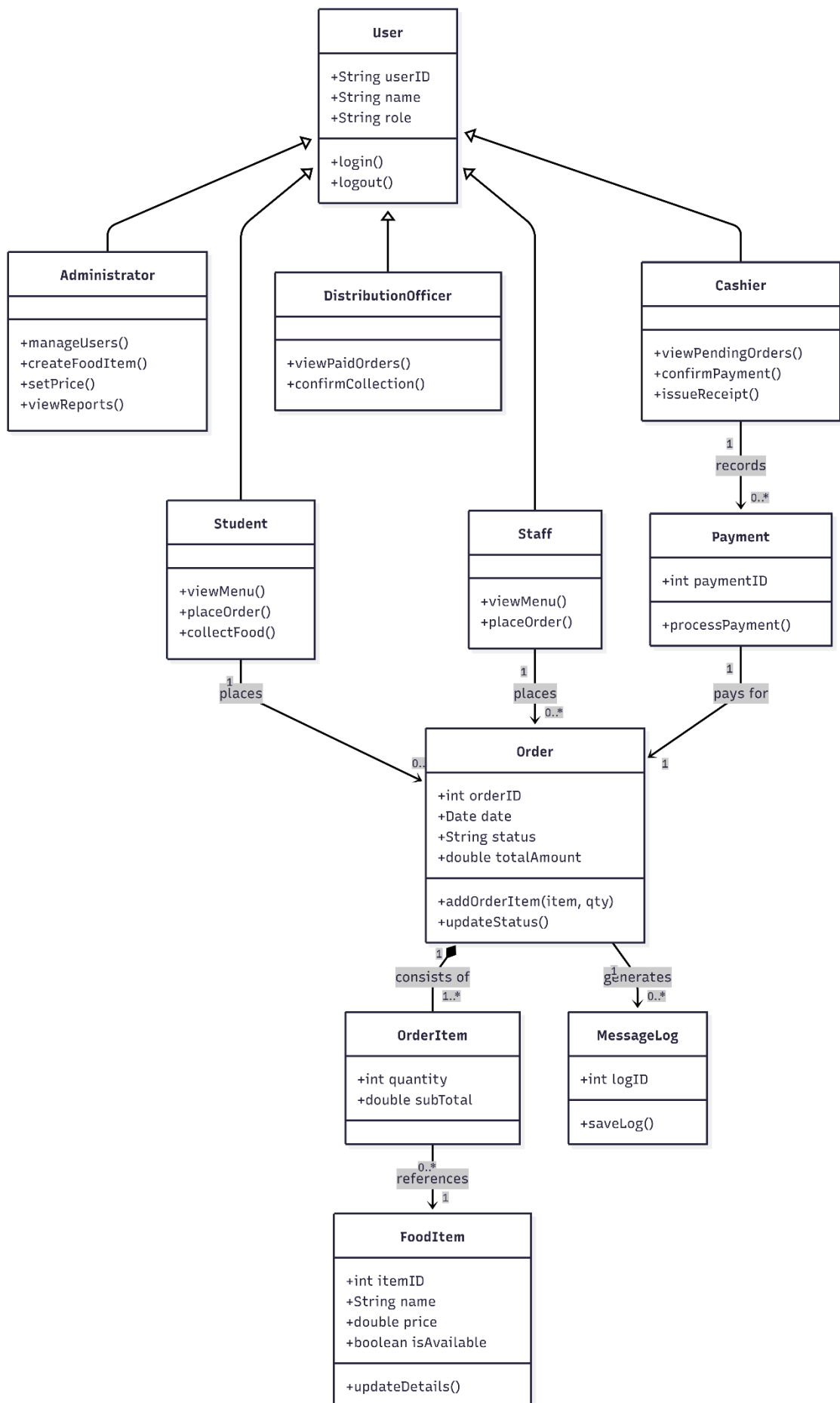
Design Explanation & Justification

The Class Diagram reflects the Object-Oriented structure of the system.

- Inheritance (User Superclass):** The system has five distinct users: Administrator, Student, Staff, Cashier, and Distribution Officer. Since they all share common attributes (ID, Name, Login), I utilized **Inheritance**. A base User class holds the

common data, while subclasses contain specific methods (e.g., Cashier has `confirmPayment`, Admin has `createFoodItem`). This promotes code reusability.

- **Staff Capability:** The requirements state that Staff "Can also place food orders like students". Consequently, the Staff class mimics the Student class structure regarding the `placeOrder()` method.
- **Composition (Order and OrderItem):** A Composition relationship is used between Order and OrderItem. An OrderItem cannot exist without an Order. This structure ensures that when an order is created, the specific quantities of food items are tightly bound to it, ensuring accurate calculation of the `totalAmount`



Entity Relationship Diagram for the canteen management system

Design Explanation & Justification

The ERD is designed to ensure data integrity and proper normalization as required by the non-functional requirements.

- **Entities:** The system identifies five core entities: USER, FOOD_ITEM, ORDER, PAYMENT, and MESSAGE_LOG.
- **Handling Quantities (ORDER_ITEM):** The requirement states the system must "Record order details, including items ordered, **quantities**, and total amount". To achieve this, I implemented a standard "Join Table" called ORDER_ITEM. This entity resolves the Many-to-Many relationship between ORDER and FOOD_ITEM and allows the system to store exactly how many of a specific item (e.g., 2 Cokes, 1 Rice) were included in a single order.
- **Accountability (MESSAGE_LOG):** The requirements explicitly state the system must "Store all sent messages in a message log". Therefore, I created a dedicated MESSAGE_LOG entity linked to the ORDER table. This ensures the school can audit exactly when a notification was sent for a specific paid order, satisfying the requirement to "keep records of important activities".
- **User Roles:** Since the system supports multiple user types (Student, Staff, Admin, etc.), the USER entity contains a role attribute to manage permissions securely

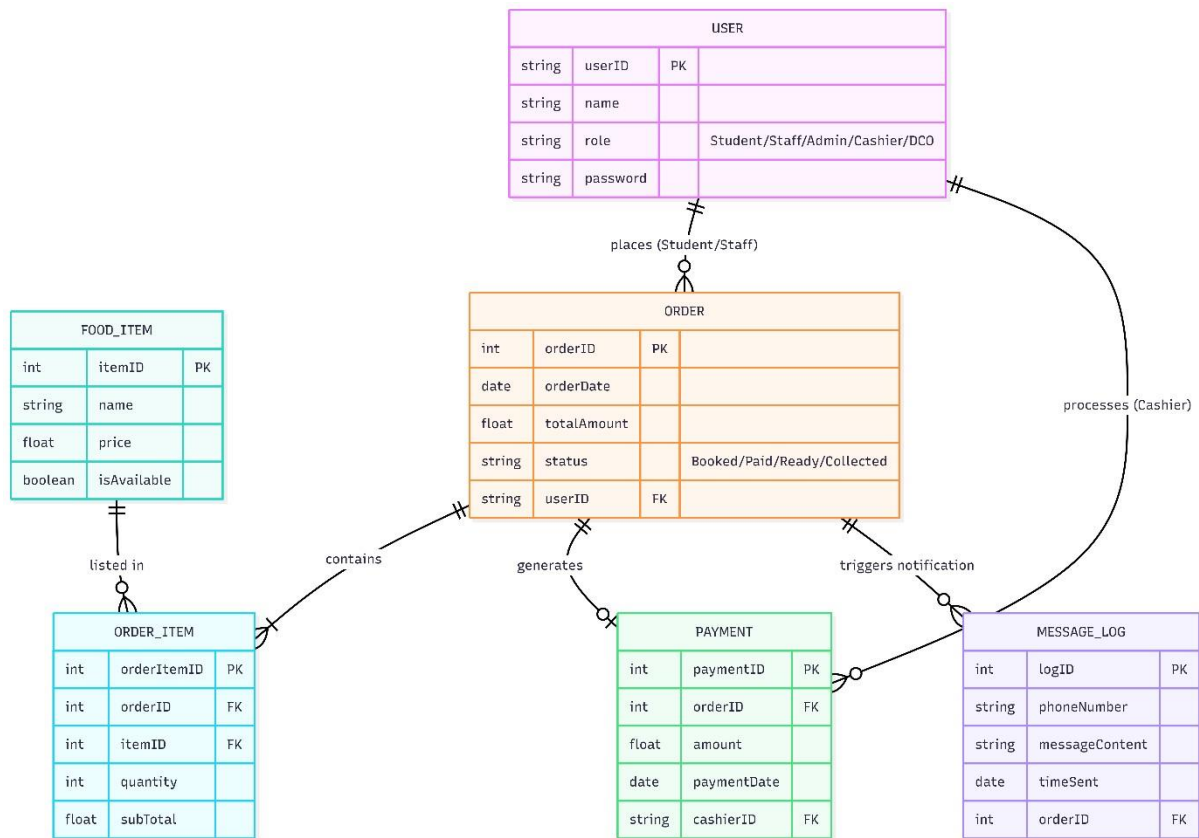


Figure 3 Entity Relationship Diagram

Sequence Diagram For The Canteen Management System

Design Explanation & Justification

This interaction diagram details the chronological flow of the most critical system process.

- **Process Flow:** The diagram initiates with the **Student** selecting items. The system calculates the total and saves the order with a status of "Booked".
- **Payment Confirmation:** The flow continues with the **Cashier**. Once they confirm payment, the diagram shows the system updating the order status to "Paid".
- **Automated Notification Logic:** Crucially, the diagram illustrates the **Parallel** process that occurs immediately after payment. The system sends a signal to the SMS Service to notify the Distribution Center.
- **Logging:** Simultaneously, the diagram shows the system saving this SMS event to the **Database (Message Log)**. This specific step is included to demonstrate compliance with the requirement to "Store all sent messages in a message log", ensuring the system meets its auditability goals.

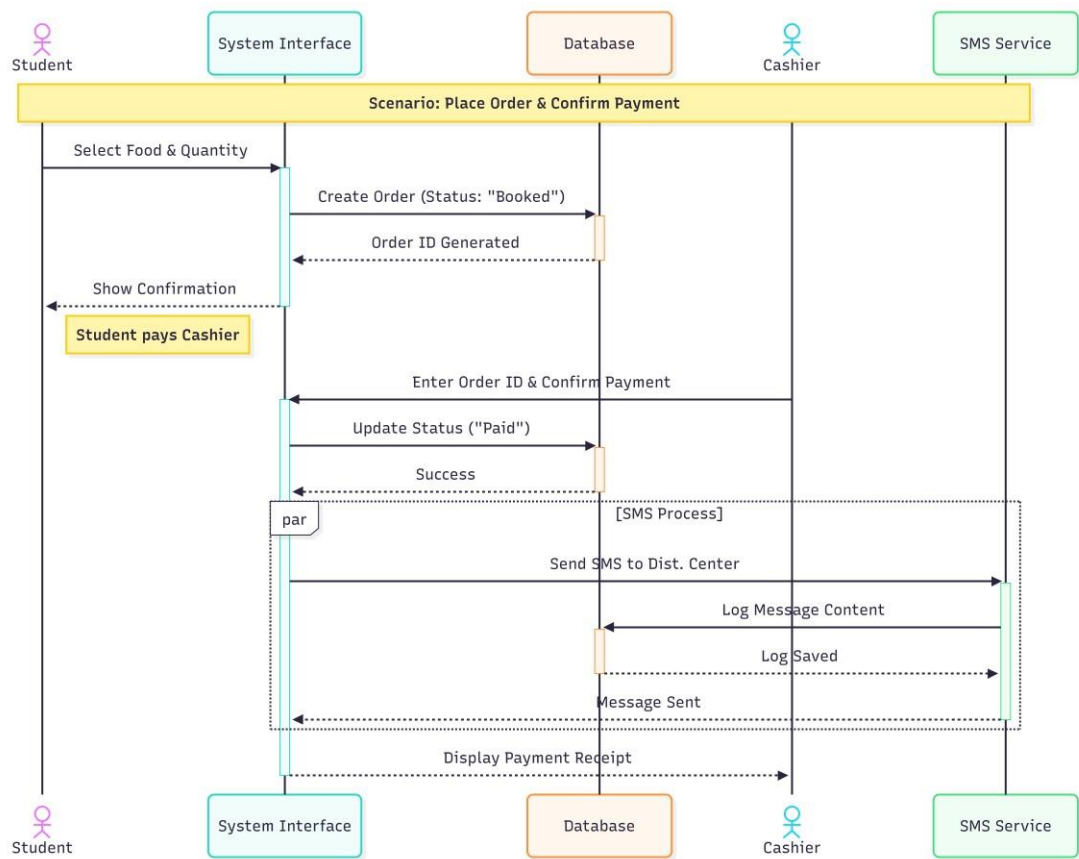


Figure 4 Sequence Diagram